Projet RCP217
IA pour des données multimédia

Grossi Valério

1er Septembre 2025
2024-2025 Semestre 2

Sommaire

1. **Introduction**
2. **Présentation de l'étude**
3. **Solution proposée**
   - 3.1 Vue d'ensemble de l'implémentation
   - 3.2 Instructions pour l'exécution du programme
   - 3.3 Entrées
   - 3.4 Sorties
     - 3.4.1 Sorties dans *Outputs* / *args.output*
     - 3.4.2 Sorties dans *TrackEval* / *data* / *trackers* / *mot_challenge*
     - 3.4.3 Sorties dans *TrackEval* / *data* / *gt* / *mot_challenge*
   - 3.5 Fichiers de données
     - 3.5.1 Fichiers d'annotations
     - 3.5.2 Fichiers de détections
       - 3.5.2.1 Présents dans le répertoire d'entrée
       - 3.5.2.2 Produits en sortie via le détecteur renseigné en CLI
     - 3.5.3 Fichiers de tracking
   - 3.6 Evaluation des sorties
4. **Exemple d'exécution**
5. **Conclusion**

1. Introduction

Dans le cadre du module *IA pour des données multimédia* RCP217, des techniques modernes d'intelligence artificielle appliquées aux données multimédias sont présentées. Le projet ici traité est « **Détection et suivi des objets dans les vidéos** » et a pour objectif de construire un pipeline pour automatiser la détection et le suivi d'objet dans des flux vidéo, ainsi que leurs évaluations.

L'objectif de l'étude et les données d'entrées utilisées sont d'abord rapidement énoncées. La solution proposée est ensuite détaillée, accompagnée de son implémentation pratique. Enfin, un exemple d'exécution du programme est fourni, suivi de l'analyse des résultats obtenus.

2. Présentation de l'étude

L'objectif des sujets assignés au thème « Multi Object Tracking » est la réalisation d'algorithmes capables d'identifier, localiser et suivre plusieurs objets au cours du temps.

Il existe de nombreuses architectures de détection et de suivi. On propose d'implémenter les plus répandues, tout en conservant une implémentation flexible afin de permettre à l'utilisateur d'intégrer facilement de nouveaux modèles.

Pour l'évaluation des différents détecteurs et trackers, des données d'entrées étiquetées sont mis à disposition par différents benchmarks dont celui *MOTChallenge* (https://motchallenge.net). C'est le formalisme de ce dernier qui est implémenté dans le projet, et pour la détection de piétons uniquement.

3. Solution proposée

3.1 Vue d'ensemble de l'implémentation

```
MultiObjectTracking/
├── Detection_models
├── Embedding_models
├── Inputs
├── MOT_main.py
├── evaluation.py
├── Tracker_models.py
└── Reid_models.py
```

Structure du répertoire avant la première exécution

Le script *MOT_main.py* a pour fonction la génération des fichiers de détections et de suivis ainsi que des images annotées correspondantes ; à partir de séquences d'images, de modèles de détection, de re-identification (embedding) et de suivi (tracking), et de leurs fichiers de poids associés.

Le script ne traite uniquement que d'entrées structurées suivant le format *MOTChallenge* (voir 3.3) et n'effectue de détections et de suivis uniquement que pour des piétons.

Le script génère ses sorties spécifiquement suivant le formalisme attendu par la librairie *TrackEval* (voir 3.4), pour son utilisation dans l'évaluation des fichiers de détections et suivis (voir 3.6). L'évaluation des sorties requiert également l'utilisation du script *evaluation.py* et est expliquée au 3.6.

Dans l'implémentation proposée :

- La famille des modèles de détections utilisables se limite à celle **YOLO** de la librairie *Ultralyics (https://docs.ultralytics.com/models/)*.

- Les modèles de suivi sont à choisir parmi **deepsort** de *deep_sort_realtime*, **bytetrack** de *supervision*, **botsort**, **strongsort**, **deepocsort**, **hybridsort** et **boostrack** de *boxmot*.

- Les fichiers de poids des modèles de re-identification, si nécessaires, sont à choisir parmi ceux spécifiés dans le dictionnaire REID_MODEL_WEIGHTS_URLS du script *Reid_models.py*.

L'utilisateur peut facilement intégrer d'autres modèles de suivi ou de re-identification, en les ajoutant simplement dans *Tracker_models.py et Reid_models.py*.

3.2 Instructions pour l'exécution du programme

Avant d'exécuter le script, le dossier d'entrée dont la structure est spécifiée au 3.3 doit être placé dans le répertoire *Inputs* et son nom doit être spécifié en argument de ligne de commande lors de l'appel au script *MOT_main.py* via --input *InputFolderName*.

Le nom du répertoire parent de celui de sortie qui sera ajouté au répertoire *Outputs (*la structure du répertoire de sortie est spécifiée au 3.4) doit également être spécifié en argument de ligne de commande lors de l'appel au script *MOT_main.py* via --output *OutputParentFolderName*.

Les noms des modèles de détection, d'embedding et de tracking que l'on souhaite utiliser doivent être spécifiés en argument de ligne de commande lors de l'appel au script :
--yolo_model *YOLOModelName1 YOLOModelName2* ...
--reid_model *ReIDModelName1 ReIDModelName2* ...
--tracker *TrackerName1 TrackerName2* ...

Plus précisément, concernant les modèles de détections et d'embedding, sont attendus en ligne de commande les noms des fichiers de poids de ces derniers, caractérisant le modèle ainsi que les valeurs spécifiques prises par ses poids (« yolov8n.pt », « osnet_x0_25_market1501.pt » et non « yolov8 », « osnet_x0_25 »).

Le script vérifie d'abord s'il existe dans *Detection_models* et *Embedding_models* des fichiers de poids aux noms assignés à yolo_model et reid_model. Si tel est le cas ce sont ces fichiers de poids qui sont utilisés.

Dans le cas contraire :

Pour chaque nom assigné à yolo_model, *Ultralytics* va le chercher dans sa liste de modèles par défaut et importer le modèle dans *Detection_models* s'il est bien défini. Pour chaque nom assigné à reid_model, le script recherche l'entrée correspondante dans le dictionnaire REID_MODEL_WEIGHTS_URLS et importe le modèle dans *Detection_models* depuis l'URL renseignée. Une erreur est renvoyée si le nom du modèle ne figure pas dans le dictionnaire ou si l'URL n'est pas valide.

Dans le cas particulier de l'utilisation du modèle de tracking deepsort, il est nécessaire d'instancier les modèles ReID dans le script plutôt que de simplement définir les chemins de leurs fichiers de poids.

Les modèles ReID instanciés sont cherchés dans *torchreid.models*. Pour chaque nom assigné à reid_model est extrait le nom du modèle ReID parmi ceux définis dans la liste DEEPSORT_REID_MODELS du script *Reid_models.py* (liste des noms de modèles ReID que supporte *torchreid.models*). Si le modèle ReID n'est pas supporté par *torchreid.models*, aucune erreur n'est levée et la combinaison ReID model + deepsort n'est pas traitée.

L'argument de ligne de commande --gen_det_images permet de générer les images de sorties annotées des détections pour les détecteurs définis.

L'argument de ligne de commande --gen_track_images permet de générer les images de sorties annotées des détections avec identifiants pour les combinaisons testées.

L'argument de ligne de commande --from_detections permet de considérer les sorties des détecteurs par défaut proposés dans le répertoire d'entrée (voir 3.3), pour les combinaisons à tester.

Les combinaisons testées proviennent d'un GridSearch, toutes les combinaisons possibles sont donc inférées.

3.3 Entrées

Le dossier de données d'entrée est à placer dans le répertoire *Inputs* et son nom doit correspondre à celui de l'argument input du parser. Ce dossier doit respecter le formalisme *MOTChallenge*.

Présentons ce formalisme au travers du dataset d'entrée MOT17, utilisé pour l'exemple et disponible à l'adresse https://motchallenge.net/data/MOT17:

```
MOT17/
├── test/
│   ├── MOT17-01-DPM/
│   │   ├── det/
│   │   │   └── det.txt
│   │   ├── img1/
│   │   │   ├── 000001.jpg
│   │   │   └── ...
│   │   └── seqinfo.ini
│   └── ...
└── train/
    ├── MOT17-02-DPM/
    │   ├── det/
    │   │   └── det.txt
    │   ├── gt/
    │   │   └── gt.txt
    │   ├── img1/
    │   │   ├── 000001.jpg
    │   │   └── ...
    │   └── seqinfo.ini
    └── ...
```

Structure du répertoire d'entrée requis

Le dossier contient plusieurs séquences d'images, leurs fichiers d'annotations, ainsi que leurs fichiers de détections générés par le détecteur par défaut propre à chaque séquence.

En effet, la compétition *MOTChallenge* propose de comparer les résultats des trackers des participants sur la base de détecteurs communs, en plus de la comparaison possible des résultats obtenus avec le pipeline (i.e. détecteur + tracker) librement implémenté (traitant directement les images).

Les détecteurs par défauts utilisés dans MOT17 sont DPM, SDP et FRCNN. Ces détecteurs sont volontairement imparfaits, probablement que chacun possède ses caractéristiques propres, ce qui permet de tester la robustesse des trackers aux détections imparfaites. Les noms de ces détecteurs sont présents dans le nom de la séquence et les résultats des détections pour chaque séquence sont contenus dans un fichier det.txt. La structure attendue de ces fichiers de détections det.txt est présentée au 3.5.2.1.

Le script *MOT_main.py* proposé traite indiféremment les séquences suivant le détecteur par défaut associé et aucun formalisme particulier n'est attendu sur le nom des séquences (*). Les fichiers de détections ne sont par ailleurs pas nécessairement attendus excepté si --from_detections est passé en ligne de commande signifiant l'intention d'effectuer du tracking sur la base des détections par défauts.

Entre le nom du dossier de données d'entrée (ici MOT17) et les séquences doit obligatoirement se trouver un niveau hierarchique contenant les répertoires *train* et/ou *test* (le script proposé traitera des deux si les deux sont présents ou un seul sinon). Les noms attendus des répertoires d'annotations, de détections et d'images sont *gt*, *det* et *img1*. Il est fortement recommandé de nommer les images numériquement afin de préserver leur ordre.

Les fichiers seqinfo.ini ne sont pas nécessairement attendus excepté si --from_detections est passé en ligne de commande. Dans ce cas, l'information du nombre d'images de la séquence y sera prélevée, indépendamment de la présence ou non du répertoire complet d'images de la séquence. Ces répertoires d'images ne sont d'ailleurs pas requis dans ce cas, excepté si --gen_det_images et/ou --gen_track_images sont passés en ligne de commande.

(*) Les noms des séquences présentes dans *train* et *test* doivent etre différents auquel cas les séquences traitées en second risquent d'effacer les premières. C'est généralement le cas pour les données d'entrées de *MOTChallenge*.

Toute les structures des fichiers textes de données d'entrées sont définies au 3.5.

3.4 Sorties

Notons args.input et args.output les arguments du parser utilisé dans le script, dont on peut spécifier en ligne de commande les valeurs (valent MOT17 et runs par défaut).

Les sorties de *MOT_main.py* sont générées à deux endroits différents, dans un sous-répertoire de *Outputs / args.output* ainsi que dans des sous-répertoires de *TrackEval / data / trackers / mot_challenge*.

Les sorties présentes dans *TrackEval* sont pour la plupart des répliques de celles contenues dans *Outputs*, mais organisées selon le formalisme standard requis par la librairie d'évaluation de Tracking TrackEval, plus spécifiquement ici pour le traitement de données au format mot_challenge.

Le script *MOT_main.py* génère également dans *TrackEval / data / gt / mot_challenge* des répertoires contenant des copies directes de données d'entrées contenues dans le répertoire args.input (fichiers d'annotations et de détections), toujours suivant le formalisme standard requis par la librairie TrackEval.

Toute les structures des fichiers textes de données de sorties sont définies au 3.5.

3.4.1 Sorties dans *Outputs / args.output*

On note *datetime* la date d'exécution du script *MOT_main.py* ayant généré les sorties en question (en chiffres année-mois-jour_heure-minute-seconde).

Dans *Outputs / args.output*, les sorties se trouvent dans le répertoire *args.input_datetime*. Ce répertoire contient les sous-répertoires de sortie associés à chaque séquence présente dans le répertoire d'entrée *args.input* (via train et test), et portant le même nom.

Chacun de ces sous-répertoires propres à une séquence est décomposé en dossiers propres au détecteur utilisé (*yolov5n*, *yolov8x*, ...), et portant le même nom. En particulier, si l'on utilise aussi le détecteur par défaut i.e. si --from_detections est passé en ligne de commande, les résultats pour ce détecteur seront contenus dans un dossier nommé « det » (le nom du détecteur par défaut est généralement inscrit dans le nom de la séquence).

Dans chaque dossier propre au détecteur utilisé se situe un sous-dossier nommé « detections » et contenant les fichiers de résultat des détections du détecteur, un par image et portant le même nom que ces dernières mais avec l'extension .txt. Dans le cas où l'on traite du détecteur par défaut, on trouvera dans le sous-dossier « detections » un unique fichier de résultat des détections de toutes les images de la séquence, nommé « det.txt », qui est une simple copie du fichier de détection présent dans le répertoire d'entrée et portant le même nom.

On trouve également dans chaque dossier propre au détecteur utilisé, si ce dernier n'est pas celui par défaut, un dossier « boxes » contenant le fichier de détection « boxes.txt » propre au détecteur en question et pour toute la séquence considérée.

Dans le cas où --gen_det_images est passé en ligne de commande, chaque dossier propre au détecteur utilisé contient un dossier d'images « img1 » contenant les mêmes images que celles présentes dans le dossier d'entrée mais annotées des résultats des détections.

Chaque dossier propre au détecteur utilisé est ensuite également décomposé en dossiers propres au tracker utilisé (*deepsort*, *bytetrack*, *botsort*, ...), et portant le même nom. On distingue les trackers n'utilisant pas de modèle de re-identification (dans cette implémentation *bytetrack*) des autres trackers.

Dans le premier cas, le dossier propre au tracker utilisé contiendra directement un dossier de résultats des suivis nommé « tracking », comportant plusieurs fichiers, un par image, et portant le même nom que ces dernières mais avec l'extension .txt. Aussi, dans le cas où --gen_track_images est passé en ligne de commande, chaque dossier propre au tracker utilisé possède un dossier d'images « img1 » contenant les mêmes images que celles présentes dans le dossier d'entrée mais annotées des résultats du tracking.

Dans le cas où le tracker utilise un modèle de re-identification, le dossier propre au tracker est alors décomposé en sous-dossiers propres au modèle ReID utilisé

(osnet_x0_25_market1501, mobilenetv2_x1_0_market1501.pt, ...). Chacun de ces sous-dossiers contient alors également comme spécifié dans l'autre cas le dossier de résultats des suivis nommé « tracking » et possiblement le dossier d'images annotées « img1 » pour le tracker en question via le modèle de ReID considéré. Dans le cas particulier de l'utilisation du tracker deepsort, et pour tout sous-dossier propre au modèle de re-identification utilisé, se trouve un dossier « embeddings » dans lequel est présent un fichier « embeddings.npy » constituant une des entrées de ce tracker.

```
Outputs/
└── runs/
    ├── MOT17_2025-08-20_19-30-45/
    │   ├── MOT17-02-DPM/
    │   │   ├── det/
    │   │   │   ├── detections/
    │   │   │   │   └── det.txt
    │   │   │   ├── img1/
    │   │   │   │   ├── 000001.jpg
    │   │   │   │   └── ...
    │   │   │   ├── bytetrack/
    │   │   │   │   ├── tracking/
    │   │   │   │   │   ├── 000001.txt
    │   │   │   │   │   └── ...
    │   │   │   │   └── img1/
    │   │   │   │       ├── 000001.jpg
    │   │   │   │       └── ...
    │   │   │   ├── deepsort/
    │   │   │   │   ├── osnet_x0_5_market1501/
    │   │   │   │   │   ├── embeddings/
    │   │   │   │   │   │   └── embeddings.npy
    │   │   │   │   │   ├── tracking/
    │   │   │   │   │   │   ├── 000001.txt
    │   │   │   │   │   │   └── ...
    │   │   │   │   │   └── img1/
    │   │   │   │   │       ├── 000001.jpg
    │   │   │   │   │       └── ...
    │   │   │   │   └── ...
    │   │   │   ├── botsort/
    │   │   │   │   ├── osnet_x0_5_market1501/
    │   │   │   │   │   ├── tracking/
    │   │   │   │   │   │   ├── 000001.txt
    │   │   │   │   │   │   └── ...
    │   │   │   │   │   └── img1/
    │   │   │   │   │       ├── 000001.jpg
    │   │   │   │   │       └── ...
    │   │   │   │   └── ...
    │   │   │   └── ...
    │   │   ├── yolov8n/
    │   │   │   ├── detections/
    │   │   │   │   ├── 000001.txt
    │   │   │   │   └── ...
    │   │   │   ├── img1/
    │   │   │   │   ├── 000001.jpg
    │   │   │   │   └── ...
    │   │   │   ├── boxes/
    │   │   │   │   └── boxes.txt
    │   │   │   ├── bytetrack/
    │   │   │   │   ├── tracking/
    │   │   │   │   │   ├── 000001.txt
    │   │   │   │   │   └── ...
    │   │   │   │   └── img1/
    │   │   │   │       ├── 000001.jpg
    │   │   │   │       └── ...
    │   │   │   ├── deepsort/
    │   │   │   │   ├── osnet_x0_5_market1501/
    │   │   │   │   │   ├── embeddings/
    │   │   │   │   │   │   └── embeddings.npy
    │   │   │   │   │   ├── tracking/
    │   │   │   │   │   │   ├── 000001.txt
    │   │   │   │   │   │   └── ...
    │   │   │   │   │   └── img1/
    │   │   │   │   │       ├── 000001.jpg
    │   │   │   │   │       └── ...
    │   │   │   │   └── ...
    │   │   │   ├── botsort/
    │   │   │   │   ├── osnet_x0_5_market1501/
    │   │   │   │   │   ├── tracking/
    │   │   │   │   │   │   ├── 000001.txt
    │   │   │   │   │   │   └── ...
    │   │   │   │   │   └── img1/
    │   │   │   │   │       ├── 000001.jpg
    │   │   │   │   │       └── ...
    │   │   │   │   └── ...
    │   │   │   └── ...
    │   │   └── ...
    │   └── ...
    └── ...
```

Structure du répertoire de sortie dans Outputs / args.output

3.4.2 Sorties dans *TrackEval / data / trackers / mot_challenge*

Dans *TrackEval / data / trackers / mot_challenge*, les sorties se situent dans les sous-répertoires *args.input-train* et/ou *args.input-test*, selon la présence ou non des répertoires *train* et *test* dans le répertoire d'entrée *args.input*.

Ces deux sous-répertoires contiennent plusieurs dossiers résultats propres aux combinaisons testées (détecteur seul / détecteur + tracker / détecteur + modèle ReID + tracker). Les combinaisons en question figurent dans le nom de ces dossiers auquel on rajoute le suffixe *datetime*.

Les fichiers de détections du détecteur par défaut étant notamment déjà présents dans le répertoire d'entrée ainsi que dans le répertoire *Output / args.output*, les dossiers résultats associés ne sont pas ici regénérés.

Quelques exemples de dossiers résultats pouvant être générés :

- det_bytetrack_2025-08-20_19-30-45
- det_osnet_x0_25_market1501_deepsort_2025-08-20_19-30-45
- det_osnet_x0_25_market1501_botsort_2025-08-20_19-30-45
- det_mobilenetv2_x1_0_market1501_deepsort_2025-08-20_19-30-45
- det_mobilenetv2_x1_0_market1501_botsort_2025-08-20_19-30-45
- yolov8n_2025-08-20_19-30-45
- yolov8n_bytetrack_2025-08-20_19-30-45
- yolov8n_osnet_x0_25_market1501_deepsort_2025-08-20_19-30-45
- yolov8n_osnet_x0_25_market1501_botsort_2025-08-20_19-30-45
- yolov8n_mobilenetv2_x1_0_market1501_deepsort_2025-08-20_19-30-45
- yolov8n_mobilenetv2_x1_0_market1501_botsort_2025-08-20_19-30-45

Chacun de ces dossiers contient un sous-dossier « data » contenant les fichiers de détection (si détecteur seul) ou de tracking (autres combinaisons) propres aux séquences testées (celles présentes dans train et test du répertoire d'entrée).

Chaque fichier de détection ou tracking est propre à toute une séquence considérée, et le fichier porte le nom de cette dernière avec l'extension .txt.

3.4.3 Sorties dans *TrackEval / data / gt / mot_challenge*

Dans *TrackEval / data / gt / mot_challenge* est copié le contenu du répertoire d'entrée *args.input* (excepté les dossiers d'images, volumineux). Ainsi, *TrackEval / data / gt / mot_challenge* contient les sous-répertoires *train* et/ou *test* (renommés *args.input-train* et *args.input-test*). Et dans ces sous-répertoires figurent les dossiers au nom des séquences, comportant notamment pour ceux présents dans *args.input-train* le dossier d'annotations « gt », ainsi que possiblement le dossier des détections par défaut « det » si --from_detections est ajouté en ligne de commande.

Dans *TrackEval / data / gt / mot_challenge,* le script *MOT_main.py* génère également un sous-répertoire « seqmaps » contenant deux fichiers *args.input-train*.txt et *args.input-test*.txt en fonction de la présence ou non des répertoires train et test dans le répertoire d'entrée *args.input*. Ces fichiers contiennent la liste des noms des séquences présentes dans le répertoire d'entrée et définies dans le split correspondant et sont nécessaires à la bonne exécution de *run_mot_challenge.py* (voir 3.6).

```
· TrackEval/
  └── data/
      ├── gt/
      │   └── mot_challenge/
      │       ├── MOT17-test/
      │       │   ├── MOT17-01-DPM/
      │       │   │   ├── det/
      │       │   │   │   └── det.txt
      │       │   │   └── seqinfo.ini
      │       │   └── ...
      │       ├── MOT17-train/
      │       │   ├── MOT17-02-DPM/
      │       │   │   ├── det/
      │       │   │   │   └── det.txt
      │       │   │   ├── gt/
      │       │   │   │   └── gt.txt
      │       │   │   └── seqinfo.ini
      │       │   └── ...
      │       └── seqmaps/
      │           ├── MOT17-test.txt
      │           └── MOT17-train.txt
      └── trackers/
          └── mot_challenge/
              ├── MOT17-test/
              │   └── ...
              └── MOT17-train/
                  ├── det_for_eval/
                  │   └── data/
                  │       ├── MOT17-02-DPM.txt
                  │       └── ...
                  ├── det_bytetrack_2025-08-20_19-30-45/
                  │   └── data/
                  │       ├── MOT17-02-DPM.txt
                  │       └── ...
                  ├── det_osnet_x0_25_market1501_deepsort_2025-08-20_19-30-45/
                  │   └── data/
                  │       ├── MOT17-02-DPM.txt
                  │       └── ...
                  ├── ...
                  ├── det_..._..._2025-08-20_19-30-45
                  ├── ...
                  ├── yolov8n_2025-08-20_19-30-45/
                  │   └── data/
                  │       ├── MOT17-02-DPM.txt
                  │       └── ...
                  ├── yolov8n_2025-08-20_19-30-45_for_eval/
                  │   └── data/
                  │       ├── MOT17-02-DPM.txt
                  │       └── ...
                  ├── yolov8n_bytetrack_2025-08-20_19-30-45/
                  │   └── data/
                  │       ├── MOT17-02-DPM.txt
                  │       └── ...
                  ├── yolov8n_osnet_x0_25_market1501_deepsort_2025-08-20_19-30-45/
                  │   └── data/
                  │       ├── MOT17-02-DPM.txt
                  │       └── ...
                  ├── ...
                  ├── yolov8n_..._..._2025-08-20_19-30-45
                  ├── ...
                  ├── ...
                  └── ...
```

Structure des sorties dans TrackEval

3.5 Fichiers de données

On présente ici les différentes variables stockées dans les fichiers d'annotations, de détection et de tracking. On associe sur chaque image un axe horizontal (de gauche à droite) et vertical (de haut en bas) dont l'unité est le pixel. On note bbox pour désigner boite englobante.

Quel que soit le type de fichier, et que ce dernier caractérise uniquement les détections d'une seule image ou de toute une séquence, chaque ligne caractérise une seule détection.

Voici les variables définies :

- **frame** : numéro de l'image
- **id** : identifiant de l'objet suivi
- **bbox-left** : valeur en abscisse du coin supérieur gauche de la bbox
- **bbox-top** : valeur en ordonnée du coin supérieur gauche de la bbox
- **bbox-right** : valeur en abscisse du coin inférieur droit de la bbox
- **bbox-down** : valeur en ordonnée du coin inférieur droit de la bbox
- **bbox-width** : largeur (en pixel) de la boite englobante
- **bbox-height** : hauteur (en pixel) de la boite englobante
- **conf** : score de confiance de la détection
- **x**, **y**, **z** : coordonnées 3D de l'objet
- **class** : classe de l'objet
- **visibility** : taux de visibilité de l'objet
- **validity flag** : indicateur de prise en compte de la détection pour l'évaluation (pour MOT17, elle est directement liée à la valeur classe)

3.5.1 Fichiers d'annotations

Les fichiers d'annotations présents dans les dossiers d'entrées de *MOTChallenge* comme celui MOT17 présentent la structure suivante :

Chaque fichier d'annotation est propre à toute une séquence, les lignes sont ordonnées suivant les identifiants repérés et chaque ligne caractérise une détection suivant frame (int), id (int), bbox-left (int), bbox-top (int), bbox-width (int), bbox-height (int), validity flag (bool), class (int), visibility (float / int).

On remarque que lorsque la classe est différente de 1 (piétons), le flag de validité vaut toujours 0 et la visibilité se voit attribuer l'entier 1 comme valeur par défaut.

3.5.2 Fichiers de détections

3.5.2.1 Présents dans le répertoire d'entrée

Les fichiers de détections présents dans le répertoire d'entrée de MOTChallenge comme celui MOT17 présentent la structure suivante :

Chaque fichier de détection est propre à toute une séquence, les lignes sont ordonnées suivant l'ordre des images et chaque ligne caractérise une détection suivant frame (int), id (int), bbox-left (float), bbox-top (float), bbox-width (float), bbox-height (float), conf (float), x (int), y (int), z (int).

Les détections sont associées aux piétons uniquement, et x, y et z valent –1 par défaut puisque le benchmark de suivi multi-objets MOTChallenge est 2D. Il s'agit de détection uniquement et non de tracking donc id vaut par défaut -1.

3.5.2.2 Produits en sortie via le détecteur renseigné en CLI

Les fichiers de détections associés aux détecteurs renseignés en ligne de commande, et qui sont présents dans le répertoire de sortie *Outputs,* sont tous propres aux détections repérées sur chaque image individuelle. Ceux présents dans le répertoire de sortie *TrackEval* sont propres aux détections repérées sur toute la séquence.

Ces fichiers de détections présentent la structure suivante :
Chaque ligne caractérise une détection suivant frame (int), id (int), bbox-left (float), bbox-top (float), bbox-width (float), bbox-height (float), conf (float), x (int), y (int), z (int).

Les détections sont associées aux piétons uniquement. Il s'agit de détection uniquement et non de tracking donc id vaut par défaut -1 et x, y et z valent –1 par défaut puisque le benchmark de suivi multi-objets MOTChallenge est 2D.

Le script génère également dans le répertoire de sortie Outputs des fichiers de détections propres aux séquences entières nommés « boxes.txt » et ne contenant uniquement frame (float), bbox-left (float), bbox-top (float), bbox-width (float), bbox-height (float).

3.5.3 Fichiers de tracking

Les fichiers de tracking présents dans le répertoire de sortie *Outputs* sont tous propres aux détections repérées sur chaque image individuelle et ceux présents dans le répertoire *TrackEval* sont propres aux détections repérées sur toute la séquence.

Ces fichiers de détections présentent la structure suivante :

Chaque ligne caractérise une détection suivant frame (int), id (int), bbox-left (float), bbox-top (float), bbox-width (float), bbox-height (float), conf (int), x (int), y (int), z (int).

Les détections concernent exclusivement les piétons, car les données en entrée du tracker se limitent à cette classe d'objet. Le score de confiance est fixé par défaut à 1 pour la majorité des trackers car non calculé. Cette valeur n'est pas utilisée par TrackEval pour l'évaluation des trackers mais la colonne doit néanmoins être présente afin de respecter le format MOTChallenge (son utilité réside dans l'évaluation des détections seules).

3.6 Evaluation des sorties

On propose d'utiliser les scripts de la librairie TrackEval pour évaluer les trackers, les différents pipelines testés et même les détecteurs seuls via une astuce mentionnée dans la suite. A la première exécution de *MOT_main.py,* la librairie TrackEval est donc téléchargée (https://github.com/JonathonLuiten/TrackEval).

On propose par exemple d'utiliser le script *run_mot_challenge.py* présent au chemin *TrackEval / scripts / run_mot_challenge.py*. Ce script calcule différents scores comme HOTA, DetA, AssA et d'autres.

Bien que la librairie TrackEval et ses scripts fonctionnent sur des fichiers de tracking uniquement, on pourrait toutefois évaluer les détecteurs seuls à partir de leurs fichiers de sorties, de détections. L'astuce est implémentée dans le script *evaluation.py* et consiste à remplacer les valeurs -1 de la colonne ID des fichiers de détection par des valeurs entières quelconques strictement positives. Le fichier de détection modifié ainsi déguisé en fichier de tracking pourra etre évalué par TrackEval, et les scores de détections seules (DetA, DetRe, DetPr) seront bien accessibles.

Le script *evaluation.py* permet ainsi d'ajouter dans *TrackEval / data / trackers / mot_challenge / args.input-split* (chemins à spécifier dans *evaluation.py*) des dossiers résultats de détections uniquement qui seront utilisables pour évaluation par les scripts de TrackEval. Le suffixe *_for_eval* est alors ajouté au nom du dossier résultat de détection.

On présente ci-dessous quelques exemples de dossiers résultats de détection pouvant etre créés via l'exécution de *evaluation.py* :

- det_for_eval
- yolov8n_2025-08-20_19-30-45_for_eval

Voici un exemple de commande (présentée également dans *evaluation.py*) pour l'exécution de *run_mot_challenge.py* utilisé pour l'obtention des métriques d'intérêt :

python TrackEval/scripts/run_mot_challenge.py --BENCHMARK MOT17 --SPLIT_TO_EVAL train --TRACKERS_TO_EVAL det_for_eval yolov8n_2025-08-20_19-30-45_for_eval det_bytetrack_2025-08-20_19-30-45 yolov8n_resnet50_dukemtmcreid_botsort_2025-08-20_19-30-45 --METRICS HOTA CLEAR Identity VACE --USE_PARALLEL True --NUM_PARALLEL_CORES 4 --OUTPUT_SUB_FOLDER results

Les données sont cherchées dans *TrackEval* / *data* / *gt* / *mot_challenge* / BENCHMARK-SPLIT_TO_EVAL et *TrackEval* / *data* / *trackers* / *mot_challenge* / BENCHMARK-SPLIT_TO_EVAL.

Les pipelines et détecteurs seuls qui sont évalués sont ceux renseignés dans TRACKERS_TO_EVAL. Toutes les séquences sont évaluées.

Les métriques calculées sont celles définies via METRICS.

USE_PARALLEL permet l'exécution sur plusieurs cœurs, NUM_PARALLEL_CORES définit le nombre de cœurs.

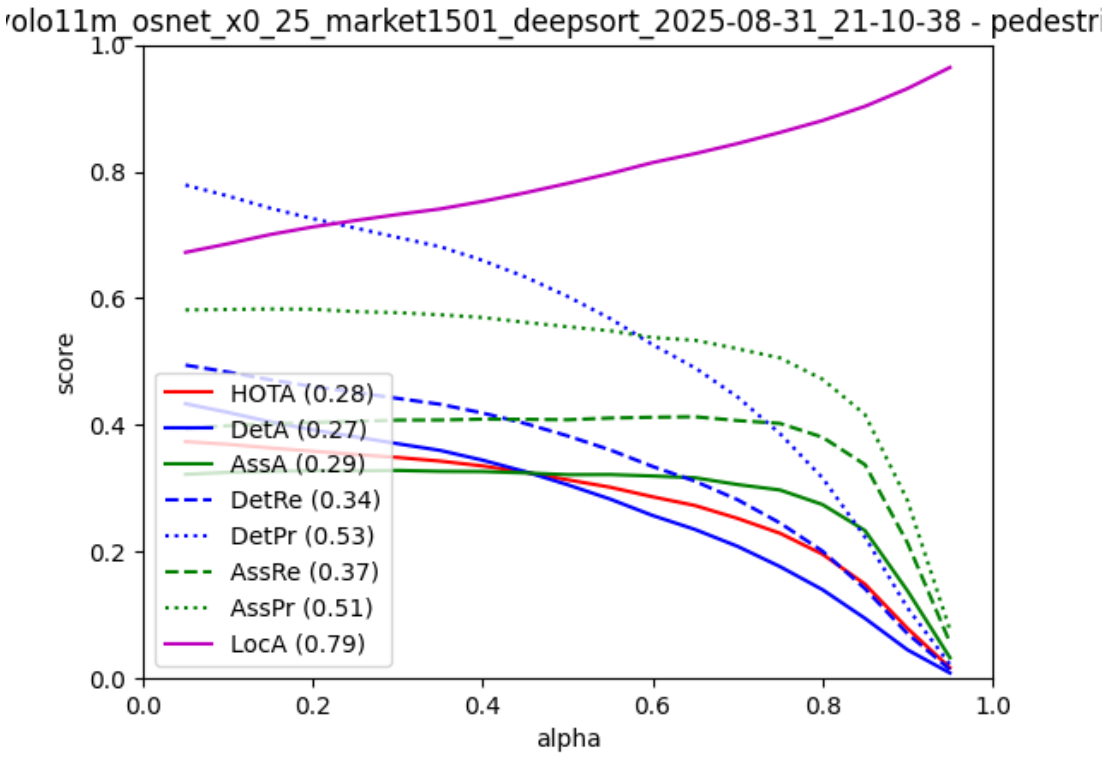
OUTPUT_SUB_FOLDER définit le nom des répertoires où sont stockés les résultats. Pour chaque pipeline ou détecteur seul testé, les résultats sont stockés dans *TrackEval* / *data* / *trackers* / *mot_challenge* / BENCHMARK-SPLIT_TO_EVAL / TRACKERS_TO_EVAL / OUTPUT_SUB_FOLDER.

4. Exemple d'exécution

On propose d'exécuter *MOT_main.py* pour l'exemple proposé dans la consigne, c'est à dire un modèle de détection YOLO et le tracker DeepSORT.

On choisit *yolo11m* d'*Ultralytics et DeepSORT de deep_sort_realtime* avec l'utilisation pour la génération des embeddings du fichier de poids modèle ReID *osnet_x0_25_market1501*.

Les séquences sur lesquelles sont calculées les métriques sont celles MOT17-02-DPM et MOT17-04-DPM de MOT17.



Graphique des métriques généré par run_mot_challenge.py

Le score LocA est élevé (79%), ce qui signifie que lorsqu'un objet est détecté, sa position dans l'image correspond bien à la vérité terrain.

En revanche, les scores de détection DetA et d'association Assa, plus faibles (respectivement 27% et 29%), attestent les difficultés du pipeline à détecter l'ensemble des piétons présents et à garder leur identité.

Bien que ces scores soient très éloignés des meilleurs résultats du benchmark, ils restent corrects. Les images annotées issues de cette combinaison sont accessibles dans le répertoire *Outputs*.

5. Conclusion

Ce projet inscrit dans le module RCP217 *IA pour des données multimédia* m'a permis d'acquérir de nombreuses notions sur les différents éléments constituant la détection et le suivi d'objets contenus dans des vidéos, ainsi que les métriques utilisées pour évaluer les résultats.

Ce projet m'a également permis de progresser dans l'intégration de plusieurs implémentations au sein d'un même pipeline, et de manière plus large de renforcer mes compétences en Python.